

Laboratory Manual: Vector Processor Design Space Exploration

Advanced Computer Architecture WT25

Univ.Prof. Dr.-Ing. Dipl.-Ing. Daniel Müller-Gritschneider
J. Parker Jones, M.Sc.

November 5, 2025

Embedded Computing Systems
TU Wien Informatics
Treitlstr. 3, 1040 Wien
Tel.: (089) 289 23666

©Further publication only with the consent of the institute. Thank you to Florian Huemer
and Heinz Deinhart.

Contents

1	Relevant Background	1
1.1	Vector Processing	1
1.2	RISC-V Vector Instruction Set	1
1.3	Vector C Intrinsic Functions	1
1.4	Vicuna2.0 Vector Co-Processor	4
1.4.1	Architecture and Timing	4
1.4.2	Simulation and Synthesis	6
2	Lab Setup	9
2.1	Accessing the Lab Material	9
2.1.1	TUWien Gitlab Access	9
2.1.2	Working on a Personal Machine	9
2.2	Repository Structure	10
2.3	Writing Your Vectorized Code	10
2.4	Modifying the Vector Processor	10
2.5	Running Simulation and Synthesis	10
3	Example: Vector Multiplication	11
3.1	Reference Program	11
3.2	Initial Vectorized Program	11
3.3	Vector Hardware Improvements	12
3.4	Further Program Improvements	12
3.5	Final Hardware Evaluation	12
4	Example: Matrix Multiplication	13
4.1	Reference Program	13
4.2	Performance and resource utilization targets	13
	Bibliography	14

1 Relevant Background

This chapter introduces relevant background information necessary to complete this Lab.

1.1 Vector Processing

Vector Processors utilize SIMD (Single Instruction Multiple Data) architecture to single operations on large vectors of independent data values. Many real-world processors utilize vector processor architectures, such as the CRAY-1 supercomputer [7], ARM NEON microprocessors[1].

The general components in a vector processor are:

- Vector Registers - wide registers where the vectors are stored, often much larger than a standard scalar register file
- Vector Functional Units - wide functional units where vector operations are performed
- Vector Load/Store Unit - dedicated memory interface for the vector system

These components are often configured in a system such as in Figure 1.1.

1.2 RISC-V Vector Instruction Set

The RISC-V V Extension defines an instruction set extension for vector operations in a RISC-V system [2]. For flexibility in implementation and code, it defines a set of design-time and run-time values that can be configured by designers. At design time, the vector register length **VLEN** is selected. Runtime configurable values are **v1**, the current vector length, **SEW** the selected element width, and **LMUL**, the register grouping multiplier.

The runtime values can be changed using the **vsetvl[i]** functions. When the requested vector length is larger than the **VLMAX** for the configuration, these instructions set **VL** = **VLMAX** and return **VLMAX**.

1.3 Vector C Intrinsic Functions

The RISC-V Vector C Intrinsics [3] provide programmers access to the features of the RISC-V Vector Extension without forcing them to write inline assembly. It defines a set of vector datatypes and functions which encode the **SEW** and **LMUL** in the datatype itself.

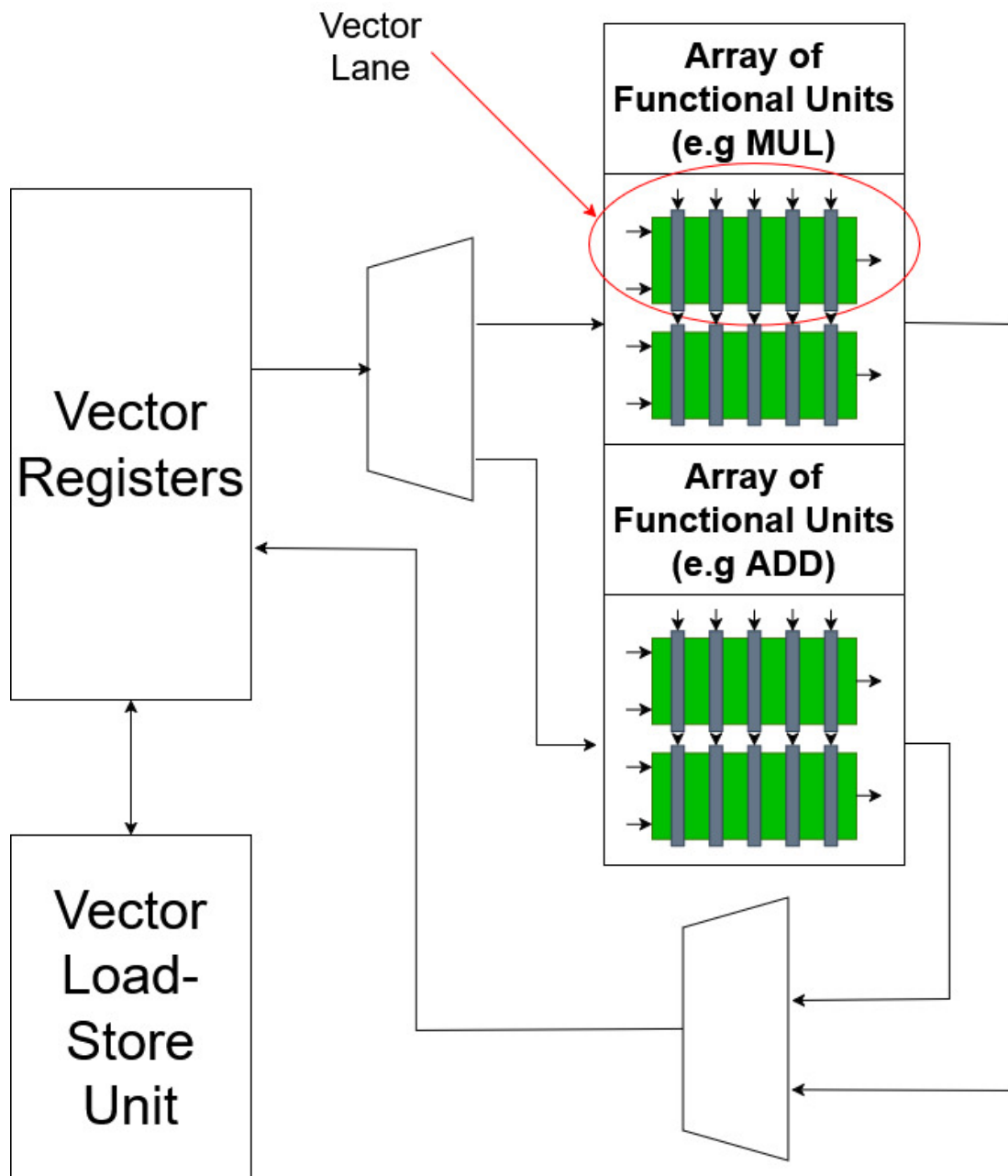


Fig. 1.1: Generic Vector Architecture

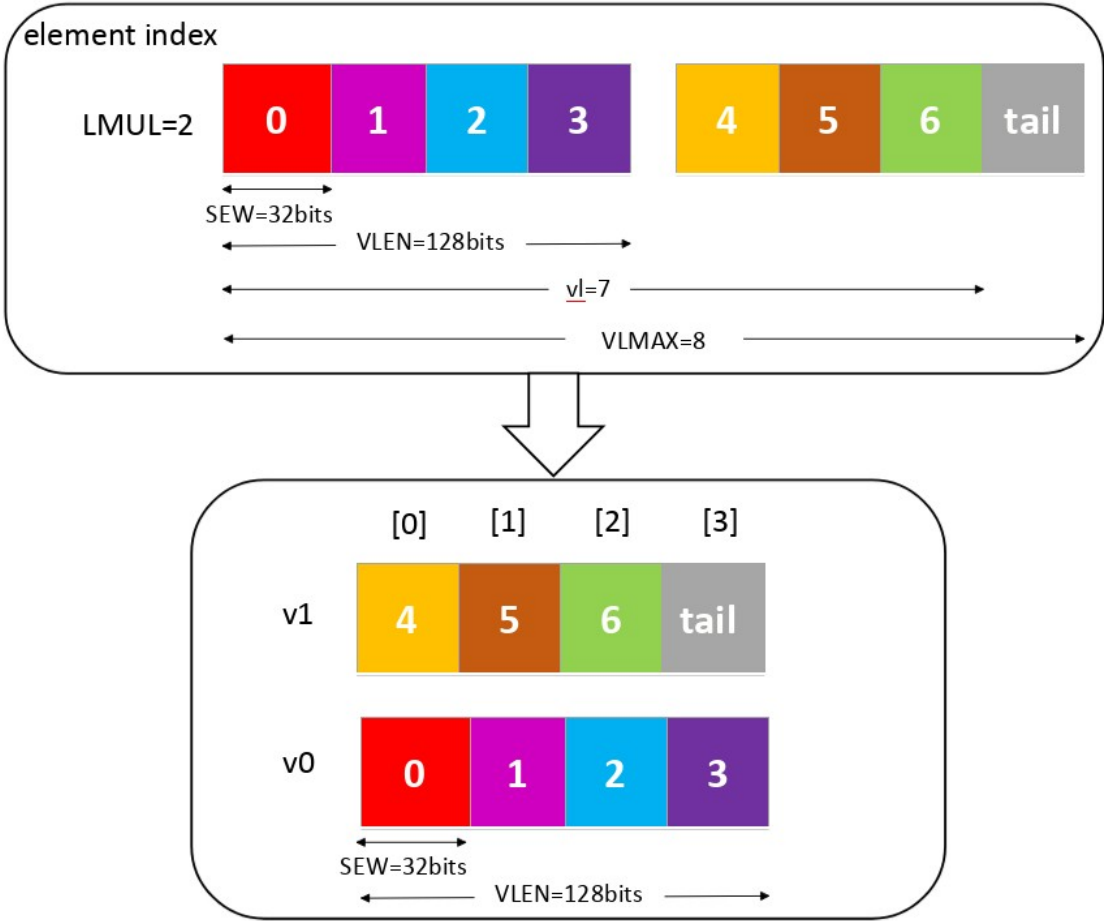


Fig. 1.2: Graphical Representation of the RISC-V Design-Time and Run-Time Parameters

```
size_t vl = __riscv_vsetvl_e8m1(16);
vint8m1_t in1 = __riscv_vle8_v_i8m1(pSrcA, vl);
vint8m1_t in2 = __riscv_vle8_v_i8m1(pSrcB, vl);
vint8m1_t output = __riscv_vadd_vv_i8m1 (in1, in2, vl);
__riscv_vse8_v_i8m1 (pDst, output, vl);
```

Fig. 1.3: Vector Add with C Intrinsic Functions

The vector intrinsics also provide access to the `vtype` and `vl` registers through a dedicated set of functions. As with the assembly variants of these functions, they return the actual length of the requested configuration in the event the requested length is too large, allowing for scalable vector code to be written. In addition, a dedicated function exists to configure the system to `VLMAX` and return this value. Examples of these functions in use can be seen in Figure 1.3.

Further information and documentation regarding the C Vector Intrinsics, including full lists of supported functions, can be found here: <https://github.com/riscv-non-isa/rvv-intrinsic-doc>.

1.4 Vicuna2.0 Vector Co-Processor

1.4.1 Architecture and Timing

The Vicuna2.0 Vector Co-Processor is an open-source implementation of a RISC-V Vector system maintained by the Embedded Computing Systems Research Unit at TU Wien [6]. It is designed to be connected to a standard scalar core such as the 4-stage CV32E40X [5]. It supports the `Zve32x` subset of the RISC-V Vector extension targeted at 32-bit embedded devices without floating-point support. In this configuration, Vicuna2.0 supports the following vector functional units with the following timing behavior for single-width operations:

- **VLSU** - Load-Store Unit, handles all memory operations - 4 cycle latency, 1 cycle initialization interval
- **VMUL** - Multiply Unit, handles multiplication and MAC - 5 cycle latency, 1 cycle initialization interval
- **VALU** - Arithmetic Logic Unit, handles arithmetic and comparisons - 3 cycle latency, 1 cycle initialization interval
- **VSLD** - Slide Unit, handles permutation operations - variable latency and initialization interval
- **VDIV** - Division Unit, handles division and remainder - 34 cycle latency, 32 cycle initialization interval

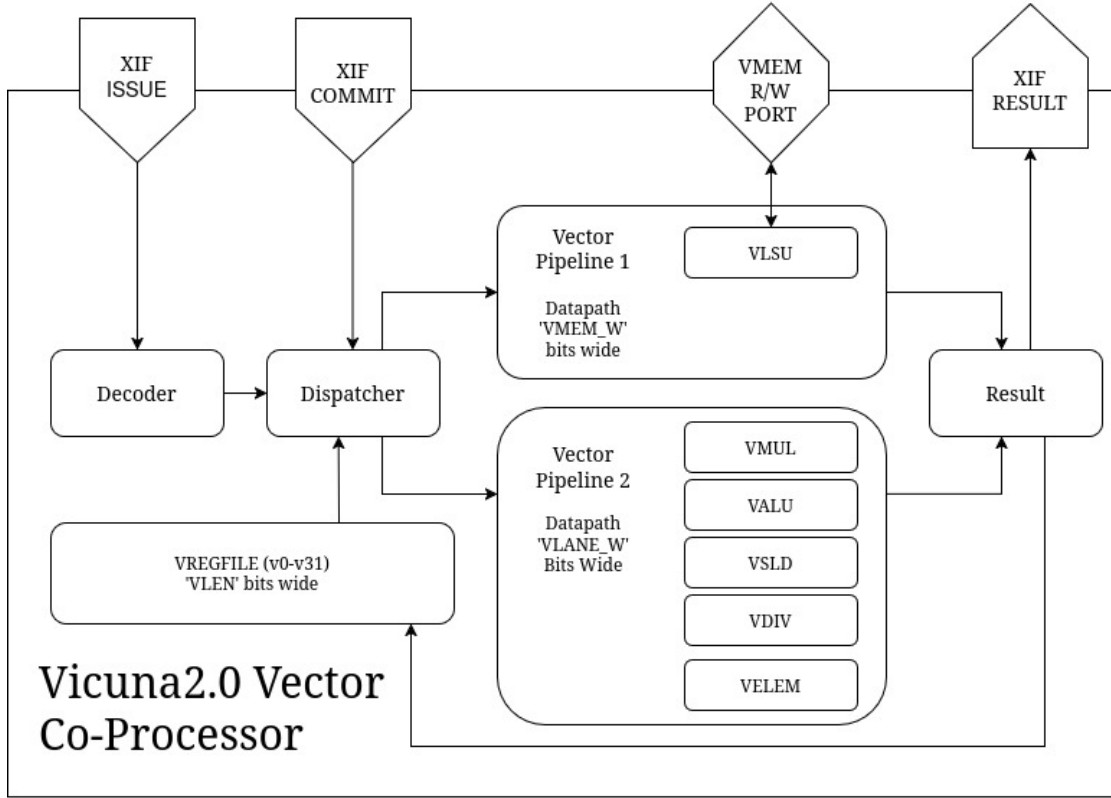


Fig. 1.4: Vicuna2.0 Vector Co-Processor

- **VELEM** - Elementwise Unit, handles reductions and operations on single vector elements - variable latency and initialization interval - Can only process 1 element per cycle

Vicuna2.0 allows the user to configure the architecture of the vector processor, allowing for a large design space. The following parameters can be configured:

- **VREG_W** - Size of Vector Register Width in bits (equal to the RISC-V **VLEN** parameter)
- **VMEM_W** - Size of the vector memory port in bits. This should always be equal to the width of the pipeline containing the **VLSU**
- Number of vector pipelines
- **PIPELINE_W** - Width of the datapath for the vector pipeline. This sets the width of all functional units within the pipeline
- Placement of vector functional units within pipelines - Vector functional units can be placed in any pipeline

These design parameters have the following restrictions and limits:

1 Relevant Background

- All 6 functional units must be present. Without all units, compliance with the Zve32x specification is not maintained. Only one functional unit of each type is allowed.
- Minimum 2 vector pipelines. Maximum 6 pipelines (1 per functional unit).
- **PIPELINE_W** is a minimum of 32 bits and a maximum of 1/2 Vector Register Width.
- For synthesis, **VMEM_W** is limited by the available I/O ports on the selected FPGA target. For simulation, the **PIPELINE_W** limit applies. See Section 1.4.2 for more details.

These parameters change the timing behavior of the vector co-processor. In particular, the following behavior is implemented:

- Wider lanes allow for functional units to process vectors faster through more parallel operations.
- For simplicity, the microarchitecture always passes entire vector register groups through the pipeline, while applying the relevant tail policy. This means that the latency of vector instructions depends only on VLMAX, independent of the currently set VL. For the best utilization, VL should be kept as close to VLMAX as possible.
- Only one vector operation can be processed in a vector pipeline at once. However, if two vector operations use different pipelines with no data dependencies, they can be processed simultaneously.
- 3 vector register read ports and 1 vector register write port are provided. For reads, whole registers are read in a single cycle, then data is shifted out to feed the functional units. For writes, outputs from the functional units are shifted in, and once a whole register is assembled, a write is performed.
- Vector operation chaining is performed at the whole-vector register level, assuming each operation uses a different pipeline.

1.4.2 Simulation and Synthesis

Vicuna2.0 is configured for simulation using Verilator [4] as well as synthesis for FPGA targets.

Verilator is an open source HDL simulator which turns VHDL, Verilog, and SystemVerilog into C/C++ which can be compiled and run on any machine. It allows for detailed data collection and testing using interfaces to standard C code and libraries.

Vicuna2.0 uses the Vivado synthesis toolchain for Xilinx FPGA targets. Synthesis imposes hardware limits on the design which are not present for simulation. Relevant to this lab are the following:

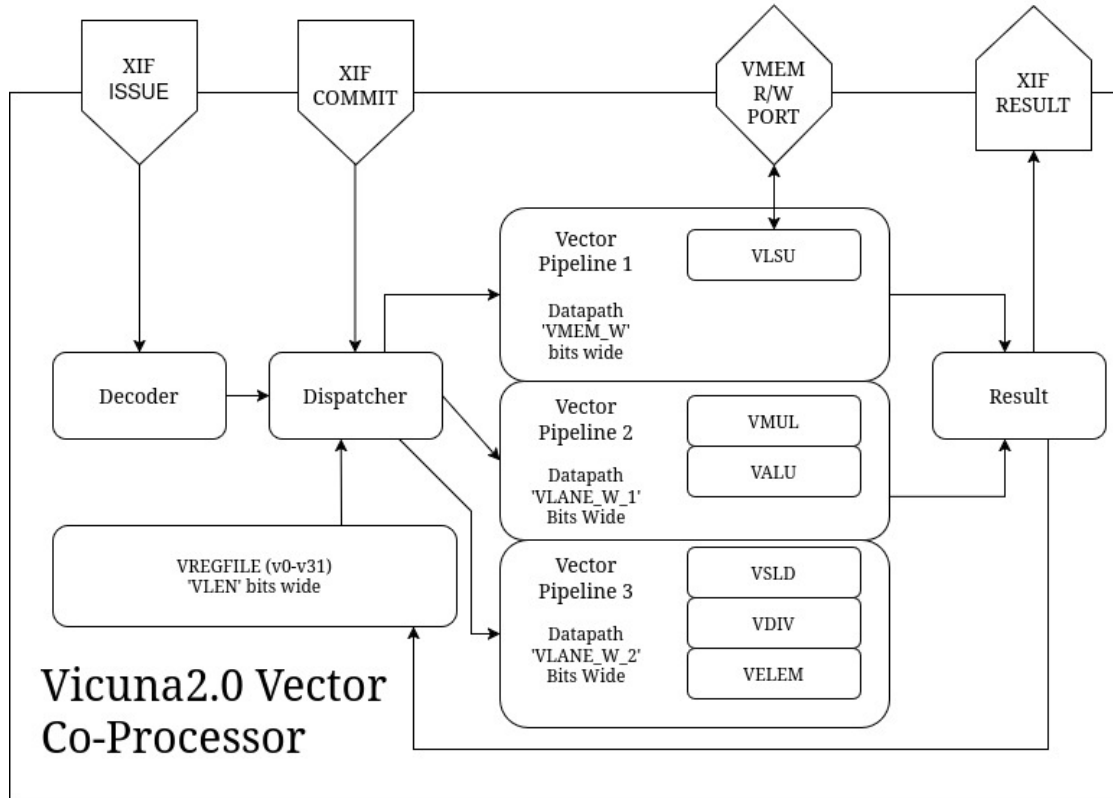


Fig. 1.5: Another valid configuration of Vicuna2.0 with 3 vector pipelines.

1 Relevant Background

- Limit on total LUT usage. Larger systems may use for LUTs than provided by the FPGA target. To complete synthesis the vector co-processor must be configured with smaller registers or functional units.
- Limit on total I/O from the FPGA. All memory interfaces are routed to the I/O pins on the FPGA, which provides an upper bound on the size of the vector memory port **VMEM_W**.

2 Lab Setup

2.1 Accessing the Lab Material

We provide three ways to access the repository containing all lab material. These are:

- **TI Lab** - You should now have a student account and access to the lab at Treitlstr. 3. Once logged in, you can clone the repository and begin work, as all dependencies are already installed under the `/opt/Vivado2019` and `/opt/hwe` directories. These paths should already be included in your `$PATH` variable.
- **Virtual Machine** - We provide a VirtualBox virtual machine with the lab repository and all dependencies installed. It can be downloaded through the TI Lab with the following command:

```
scp ${TILAB_USERNAME}@ssh.tilab.tuwien.ac.at:/opt/aca/vm_images/ACAVM1_0.tar.gz  
${STORAGE_PATH_ON_YOUR_MACHINE}
```

Keep in mind, this virtual machine is very large (50 GB download, 120 GB uncompressed).

On the virtual machine, the top level lab repo is `/Lab1_Vicuna_HWE/vicuna_edu/`.

The password for the user on the virtual machine is **1234**

- Clone the repository locally on your own machine. See section 2.1.2 for more details.

2.1.1 TUWien Gitlab Access

The lab repository is hosted on the TUWien Gitlab (<https://gitlab.tuwien.ac.at/>).

If you have not already, please fill out the survey on TUWEL with your gitlab user ID. This is NOT your Matrikelnummer or gitlab username. It can be found once you log into the TUWien gitlab under your gitlab profile settings. We require this number to give you access to the repository.

2.1.2 Working on a Personal Machine

If you wish to work natively on personal machine, you will require the RISC-V GCC cross compiler, verilator, srecord, and Vivado2019 packages to be installed. We provide a script `/setup_toolchain.sh` which can set up the first three of these dependencies, which has been tested on Ubuntu 24.04. We recommend using one of our provided environments (virtual machine or TILab machine) where these dependencies are already installed as we cannot assist with support for personal machines.

2.2 Repository Structure

The total repository contains two folders, */student_files* and */sources*. You should only modify files under the */student_files* directory.

The repository utilizes git submodules. Immediately after cloning, you should download all additional sources using the following command:

```
'git submodule update --init --recursive'
```

If you are interested in the inner workings of Vicuna2.0, the rtl sources can be found under */sources/rtl/vicuna2_core/rtl*. Detailed knowledge of the structure (outside of what is described in Chapter 1) is not required.

To assist with debugging, specialized UART print functions have been provided which route to the simulations standard output. Definitions for these can be found under */sources/programs/bsp/uart.hpp*

2.3 Writing Your Vectorized Code

Under */student_files/testcases/*, a folder is present for each test case. Your vectorized code should replace the reference code in these folders.

A duplicate of this default reference code exists under */sources/programs/testcases*. This code also compiled and run for comparison with your implementation and should not be changed.

The available test cases and their parameters can be found under */sources/programs/testcases/*/test_data*. Some test cases are provided for testing purposes, while others are expected to be sped up with vectorization.

2.4 Modifying the Vector Processor

All configuration variables for the Vicuna2.0 vector co-processor are exposed in the file */student_files/vector_config.cmake*. These are the variables described in Chapter 1.

2.5 Running Simulation and Synthesis

To assist with executing the simulation and synthesis environment, two bash scripts have been provided at the top level of the repository. These are:

- *./run_simulation.sh* - builds the verilator model, compiles the code, and executes the testcase provided in the first argument.
- *./run_synthesis.sh* - Synthesizes the simulated model for an FPGA target and reports LUT utilization.

WARNING: The *./run_synthesis.sh* script utilizes files produced by running the *./run_simulation.sh* script. To get accurate synthesis data, ensure you run a successful simulation first.

3 Example: Vector Multiplication

This chapter provides a walkthrough of the testing environment through a simple example: multiplication of two vectors. You will perform a co-design of the vector software and hardware to find a configuration within the design space provided by Vicuna2.0 which satisfies the given performance and resource utilization constraints.

A series of questions are provided with each step. We expect that you are able to answer these questions for this co-design process and apply these principles to other programs.

3.1 Reference Program

Open the program file located at `/student_files/testcases/mul/mul.cpp`.

The vector multiplication program takes in two vectors **A** and **B**. They are multiplied elementwise to produce a **C** vector output, which is widened to account for overflow.

Questions:

- What properties of this program make it a good candidate for vectorization?

3.2 Initial Vectorized Program

Write a simple vectorized program using the vector intrinsics to replace the default version of the vector multiplication function.

Once complete, run `./run_simulation.sh mul_4` to evaluate your program relative to the reference implementation. Your performance target is a **>15%** improvement in cycles for this benchmark.

Try other test cases as well to observe trends in the vector processor performance.

Questions:

- How is the vector processor achieving a speedup over the scalar reference system?
- Why is this speedup achieved for the `mul_4` benchmark, but not for the `mul_0` benchmark?
- What are the current hardware bottlenecks for your implementation?
- Why is the % improvement for cycles lower than the % improvement for instructions executed for the vector system?

3.3 Vector Hardware Improvements

Open the vector unit configuration file at `/student_files/vector_config.cmake`

With basic vectorized code written, the hardware can be modified as well. Make changes to the configuration and run the simulation again. Your performance target is a **>20%** improvement in cycles for the `mul_4` benchmark.

Questions:

- How did these changes improve the performance of the vector system?
- What features of the vector processor can your code make use of to improve performance?

3.4 Further Program Improvements

Return to your vectorized multiplication code. How could it be improved to make further use of the vector unit? Make these improvements. Your performance target is a **>40%** improvement in cycles for the `mul_4` benchmark.

Questions:

- What principles did you use when changing the vector code?
- How does your new code improve utilization of the available hardware resources?
- What is the implication of this new code on code size?

3.5 Final Hardware Evaluation

With this final system tested, use the `./run_synthesis.sh` script to synthesize your vector unit to evaluate LUT usage on the FGPA. Your resource utilization target is **< 20000** LUTs.

Questions:

- How does increasing or decreasing the parameters of the vector unit impact resource utilization?
- What are the two hard limits on this system for FPGA synthesis?

4 Example: Matrix Multiplication

This chapter introduces a more complex program for vectorization: matrix multiplication.

4.1 Reference Program

The `matmul` example performs a matrix multiplication between a **A** and **B** matrix, storing the result in a **C** matrix. Datatypes are widened to account for potential overflow.

This example provides additional complexity and possibilities for vectorization, but the principles explored in Chapter 3 still apply. When performing the design space exploration, consider the following:

- The B matrix has not been transposed. Consider the implications of accesses to this matrix.
- Reduction operations can have extremely large latencies.
- Consider the impact of matrix dimensions on optimal register size and grouping.
- Multiple nested loops provide different options for unrolling.
- For matrix multiplication, each input is used multiple times.
- This is a complex program with many optimization possibilities. Multiple valid solutions may exist which satisfy the design targets.

4.2 Performance and resource utilization targets

These targets do not represent the maximum possible speedup or minimum LUT usage, just guidelines to test if you are using the concepts discussed in lecture. Feel free to experiment!

	Cycles_1	Cycles_2	Cycles_3	LUT Usage
<code>matmul_4</code>	40%	45%	50%	16000
<code>matmul_8</code>	–	–	70%	20000

Table 4.1: Targets for Matmul Example

Bibliography

- [1] ARM. Fundamentals of armv8 neon technology. <https://developer.arm.com/documentation/102474/0100/Fundamentals-of-Armv8-Neon-technology>. Accessed: 2025-11-02.
- [2] RISC-V Collaboration. The risc-v instruction set manual, vol 1, unpriveleged architecture, version 20240411.
- [3] RISC-V Collaboration. Risc-v vector intrinsic document. <https://github.com/riscv-non-isa/rvv-intrinsic-doc/tree/main>. Accessed: 2025-11-02.
- [4] Verilator Collaboration. Verilator. <https://www.veripool.org/verilator/>. Accessed: 2025-02-07.
- [5] OpenHW Group. Openhw group core-v cv32e40x risc-v ip. <https://github.com/openhwgroup/cv32e40x>. Accessed: 2025-02-07.
- [6] Jefferson Jones, Philipp Kempen, and Daniel Mueller-Gritschneider. Vicuna2.0: Risc-v embedded vector unit with half-precision floating-point support for tinymml. pages 69–72, 09 2025.
- [7] Richard M. Russell. The cray-1 computer system. *Commun. ACM*, 21(1):63â72, January 1978.